Plan mở rộng: chữ số, chữ cái và dấu câu

Bộ tăng tốc TinyML trên FPGA (Basys 3 / Nexys A7-100T) · nâng từ 10 lớp (MNIST) lên khoảng 57 lớp · bản kế hoạch, chưa có code

1. Mục tiêu và phạm vi

Giữ nguyên kiến trúc bản 0–9 (UART → bộ đệm → các lớp fully-connected int8 → argmax), chỉ **mở rộng mô hình và số lớp đầu ra**. FPGA vẫn nhận **từng ký tự 28×28**; việc cắt một dòng chữ thành từng ký tự làm trên PC.

Giai đoạn	Nội dung	Số lớp	Tiêu chí đạt (độ chính xác là ước tính, cần đo lại)
0	Hoàn thiện bản 0–9 hiện tại	10	Chạy trên board, khớp 100 % từng bit với Python, ≥ 95 %
1	0–9 + A–Z chữ hoa	36	≥ 85 % trên tập test; phần cứng khớp 100 % với Python
2	+ chữ thường (EMNIST Balanced) + 10 dấu câu	≈ 57	≥ 80 %; dấu câu nhận đúng ≥ 85 %
3	Nhận cả dòng chữ (PC cắt ký tự, gửi lần lượt)	≈ 57	Demo viết một câu ngắn, in đúng chuỗi trên PC

> **Nguyên tắc:** giai đoạn 0 phải xong trước. Mỗi giai đoạn sau chỉ đổi hằng số trong package, thêm file trọng số và thêm 1 lớp; không viết lại các khối đã chạy đúng.

2. Dữ liệu và nhãn

Nguồn	Lớp	Ghi chú
MNIST	0–9	Đang dùng
EMNIST Letters / Balanced	A–Z; Balanced có 47 lớp (gộp các chữ hoa/thường giống nhau như c/C, o/O, s/S, x/X…)	Ảnh EMNIST bị **xoay và lật** so với MNIST, phải chuyển vị lại khi tiền xử lý. Cùng định dạng 28×28, uint8.
Tự thu thập (+ HASYv2 nếu thiếu)	10 dấu: `. , ? ! : ; - ( ) '`	Mỗi dấu ≥ 500 mẫu. Dùng một công cụ vẽ trên PC để tạo mẫu. Căn giữa theo khối lượng và phóng to giống MNIST; `.` và `,` rất dễ nhầm với ảnh trống.

Bảng nhãn (file mới `gen/labels.mem`): dòng thứ k là mã ASCII (hex) của lớp k. Ví dụ: lớp 0–9 → `30`–`39`, lớp 10 → `41` ('A'), lớp 46 → `74` ('t'), lớp 47 → `2E` ('.'). Phần cứng chỉ cần tra bảng này, không cần biết lớp nào là chữ gì.

3. Mô hình và lượng tử hoá

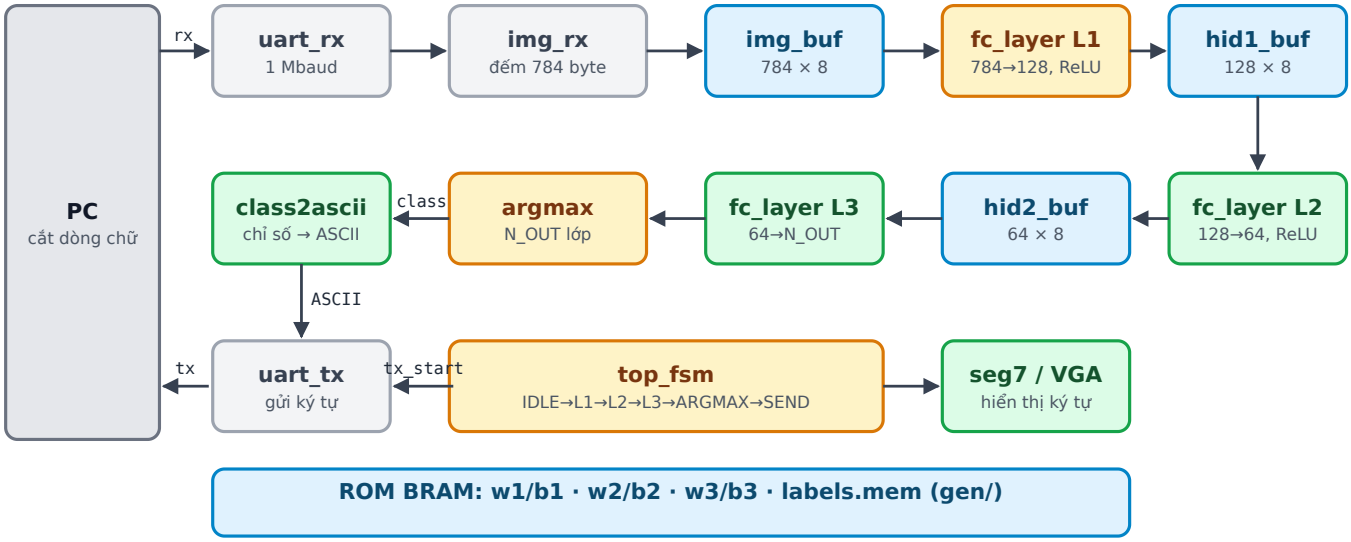
Hạng mục	Bản 0–9	Bản mở rộng
Kiến trúc	`784-32-10`	`784-128-64-N_OUT (thêm 1 lớp ẩn)`
Số tham số	≈ 25 nghìn	≈ 113 nghìn (≈ 110 KB int8)

Requant	`M1`	`M1` (lớp 1), `M2` (lớp 2); lớp ra vẫn giữ logit int32
Huấn luyện	Float rồi lượng tử	Nên dùng **QAT** (huấn luyện có lượng tử) để giảm mất độ chính xác khi số lớp nhiều

Số lớp không chia hết cho số MAC song song thì **đệm thêm neuron giả** cho đủ bội số của P: trọng số = 0 và bias = -2^{31}, để argmax không bao giờ chọn. Ví dụ 57 lớp với P3 = 8 → đệm lên 64. Bên AI xuất file đã đệm sẵn.

> **Nếu MLP không đạt mục tiêu:** phương án dự phòng là CNN nhỏ (2 lớp tích chập 3×3 + 1 lớp FC), độ chính xác ước tính ≥ 88 %. Phương án này cần thêm khối tích chập và line buffer, nên tách thành một giai đoạn riêng.

4. Sơ đồ khối mới



MỚI module viết mới SỬA module có sẵn cần sửa / tham số hoá GIỮ giữ nguyên xanh dương: bộ nhớ

5. Cấu trúc thư mục

```
tinyml-fpga/
├── README.md                      SỬA  thêm mục giai đoạn mở rộng
├── ai/
│   ├── train_quant_export.py      SỬA  thêm --dataset mnist|emnist36|emnist57, 3 lớp, QAT, đệm
neuron
│   ├── dataset_emnist.py          MỚI  tải EMNIST, xoay lại ảnh, ghép nhãn với dấu câu
│   ├── draw_collect.py            MỚI  công cụ vẽ để thu mẫu dấu câu (lưu .png 28×28)
│   ├── segment_line.py            MỚI  cắt ảnh một dòng chữ thành từng ký tự, chuẩn hoá 28×28
│   ├── send_image.py              SỬA  1 Mbaud, gửi nhiều ký tự liên tiếp, in chuỗi kết quả
│   └── cpu_baseline.py            SỬA  đo CPU cho mô hình 3 lớp
├── gen/                           (tự sinh, không sửa tay)
│   ├── w1.mem b1.mem w2.mem b2.mem    SỬA  kích thước mới
│   ├── w3.mem b3.mem              MỚI  lớp thứ 3
│   ├── labels.mem                 MỚI  chỉ số lớp → mã ASCII
│   ├── quant_params.svh           SỬA  thêm N_HID2, M2, N_OUT_PAD
│   └── test_images.mem, test_expected.mem   SỬA  mẫu đủ mọi lớp
├── rtl/
│   ├── tinyml_pkg.sv              SỬA  tham số mới (mục 6)
│   ├── top_fsm.sv                 SỬA  thêm trạng thái S_L3
│   ├── uart_rx.sv, uart_tx.sv     GIỮ
│   ├── img_rx.sv                  GIỮ  khối nhận ảnh (làm ở giai đoạn 0)
│   ├── buf_ram.sv                 MỚI  RAM đệm dùng chung, tham số DEPTH (img/hid1/hid2)
│   ├── mac_pe.sv                  GIỮ
│   ├── fc_layer.sv                SỬA  tham số hoá IN, OUT, P, file ROM, M, bật/tắt ReLU
│   ├── argmax.sv                  SỬA  tham số N_OUT, bỏ qua neuron đệm
│   ├── class2ascii.sv             MỚI  ROM labels.mem: chỉ số → ASCII
│   ├── seg7_char.sv               MỚI  hiển thị chữ/số trên LED 7 đoạn (font rút gọn)
│   ├── vga_char.sv                MỚI  (tuỳ chọn) hiện ký tự lớn ra màn hình VGA
│   ├── basys3_top.sv              SỬA  nối thêm L3, class2ascii, hiển thị
│   └── nexys_a7_top.sv            SỬA
├── tb/
│   ├── tb_uart.sv                 GIỮ
│   ├── tb_fc_layer.sv             MỚI  kiểm từng lớp riêng với vector từ Python
│   ├── tb_tinyml_accel.sv         SỬA  3 lớp, so bit-exact cho mọi lớp ký tự
│   └── tb_board_top.sv            SỬA  gửi chuỗi ký tự qua UART, kiểm chuỗi trả về
├── constr/  basys3.xdc  nexys_a7_100t.xdc     SỬA  thêm chân VGA nếu dùng
└── docs/    Plan_mo_rong_ky_tu.pdf            MỚI  tài liệu này
```

6. Thay đổi trong tinyml_pkg

Hằng số	Bản 0-9	Bản mở rộng	Ghi chú
N_HID	32	128	Lớp ẩn 1
N_HID2	—	64	Mới: lớp ẩn 2
N_OUT	10	36 / 57	Theo giai đoạn
N_OUT_PAD	—	40 / 64	Mới: N_OUT làm tròn lên bội số của PE3
PE1, PE2, PE3	8, 10, —	8, 8, 8	Có thể tăng PE1 lên 16 trên Nexys A7
OUT_IDX_W	4	6	Tự tính bằng `$clog2(N_OUT_PAD)`
M1, M2	M1	M1, M2	Bên AI cung cấp
W3_FILE, B3_FILE, LABEL_FILE	—	"w3.mem"…	Mới

`requant_relu()`	1 hệ số cố định	Thêm tham số M để dùng chung cho lớp 1 và 2

7. Module cần bổ sung và sửa

Module		Nhiệm vụ	Cổng / tham số chính
`fc_layer`	SỬA	Một module cho cả 3 lớp: đọc đầu vào từ RAM, đọc word ROM chứa P trọng số, P MAC cộng dồn, cộng bias, requant + ReLU (lớp ẩn) hoặc xuất logit (lớp ra).	param `IN, OUT, P, W_FILE, B_FILE, M, RELU`; `start, done, in_addr, in_data, out_we, out_addr, out_data`
`buf_ram`	MỚI	RAM 1 cổng ghi, 1 cổng đọc, dùng cho img_buf, hid1_buf, hid2_buf. Vivado tự chọn BRAM hoặc LUTRAM.	param `DEPTH, WIDTH`; `we, waddr, wdata, raddr, rdata`
`argmax`	SỬA	Tìm logit lớn nhất trong N_OUT lớp thật, hoà thì lấy chỉ số nhỏ. Nên so sánh dần ngay khi lớp 3 xuất từng logit.	param `N_OUT`; `in_valid, in_idx, in_logit, done, class_idx`
`class2ascii`	MỚI	ROM nhỏ đọc `labels.mem`: chỉ số lớp → mã ASCII để gửi qua UART và hiển thị.	`class_idx → ascii[7:0]`
`top_fsm`	SỬA	Thêm trạng thái `S_L3`; `tx_data` lấy từ class2ascii thay cho `'0'+digit`.	thêm `l3_start, l3_done, ascii`
`seg7_char`	MỚI	Hiện ký tự trên LED 7 đoạn: số và phần lớn chữ (A b C d E F H J L o P r t U y...); ký tự không vẽ được thì hiện mã hex.	`ascii → seg[6:0], an[3:0]`
`vga_char`	MỚI	(Tuỳ chọn) xuất VGA 640×480, vẽ ký tự lớn từ ROM font 8×8 phóng to; hai board đều có cổng VGA.	`ascii → vga_r/g/b, hsync, vsync`
`img_rx`	GIỮ	Khối nhận ảnh từ giai đoạn 0: đếm 784 byte, nghỉ quá 10 ms thì bỏ khung dở.	`rx_data, rx_valid, rx_enable → img_we, img_waddr, img_ready`
`uart_rx/tx, mac_pe`	GIỮ	Không đổi.	

8. Tài nguyên và tốc độ ước tính

Hạng mục	Bản 0–9	Bản mở rộng	Basys 3 có
ROM w1 (784×128×8 bit)	7 BRAM36	≈ 25 BRAM36	50 BRAM36: vừa, còn dư khoảng 20
ROM w2, w3, bộ đệm, labels	LUTRAM	≈ 3–4 BRAM36	
DSP (P1+P2+P3)	18	24 (hoặc 8 nếu dùng chung 1 dãy MAC)	90
Chu kỳ tính 1 ký tự	3.227	≈ 14.500 (L1 ≈ 12.700)	
Thời gian @ 100 MHz	32 µs	≈ 145 µs	
Truyền 1 ảnh @ 1 Mbaud	≈ 7,8 ms (vẫn là phần chậm nhất)		

Số liệu tính theo công thức chu kỳ ≈ (OUT/P)·(IN+P+3) của fc_layer và dung lượng 36 Kbit mỗi BRAM36; cần xác nhận bằng báo cáo Vivado.

9. Quy ước AI ↔ phần cứng bổ sung

Mục	Quy ước
Tiền xử lý	Giữ `x_q = pixel >> 1`. Ảnh EMNIST phải chuyển vị về cùng hướng MNIST **trước** khi xuất `test_images.mem`.
Requant	Lớp 1 và 2: `h = clamp((acc·Mk + 2^23) >>> 24, 0, 127)`, k = 1, 2. Lớp 3: logit int32.
Bố cục ROM	Giữ `addr = g·IN + i`, `word[8k+7:8k] = W[g·P+k][i]` cho cả 3 lớp.
Neuron đệm	Trọng số 0, bias `80000000` (-2^{31}); argmax chỉ xét N_OUT lớp đầu.
`labels.mem`	N_OUT dòng, mỗi dòng 2 ký tự hex = mã ASCII của lớp.
`test_expected.mem`	Mỗi ảnh (1 + N_OUT) dòng: chỉ số lớp dự đoán, rồi N_OUT logit int32.
UART	1.000.000 baud, 8N1. Mỗi ký tự gửi 784 byte; FPGA trả 1 byte ASCII.

10. Nhiệm vụ

#	GĐ	Nhiệm vụ	Kết quả cần có
1	0	Hoàn thiện bản 0–9: img_rx, mac_pe, fc_layer, argmax, top board, chạy trên board	Demo 0–9, bit-exact 100 %
2	1	Chốt quy ước mục 9, thêm vào README	Bảng quy ước được cả nhóm đồng ý
3	1	Chuẩn bị dữ liệu EMNIST 36 lớp, xoay ảnh, tạo `labels.mem`	`dataset_emnist.py`
4	1	Huấn luyện 784-128-64-36 (QAT), xuất w1–w3, b1–b3, M1, M2, test vector	Độ chính xác ≥ 85 %, các file trong `gen/`
5	1	Tham số hoá `fc_layer`, viết `buf_ram`, `tb_fc_layer`	Từng lớp khớp bit-exact với Python
6	1	Cập nhật package, top_fsm (S_L3), argmax (N_OUT), thêm class2ascii	`tb_tinyml_accel` PASS 3 lớp
7	1	seg7_char, cập nhật top 2 board, nạp và đo	Demo 36 ký tự trên board; báo cáo tài nguyên, timing
8	2	Thu mẫu dấu câu bằng `draw_collect.py`, ghép với EMNIST Balanced	Bộ dữ liệu khoảng 57 lớp
9	2	Huấn luyện lại 57 lớp, xuất file; phần cứng chỉ đổi N_OUT, N_OUT_PAD	≥ 80 %, bit-exact 100 %
10	3	`segment_line.py`: cắt dòng chữ thành từng ký tự; `send_image.py` gửi lần lượt	In đúng chuỗi trên PC
11	3	(Tuỳ chọn) `vga_char` hiển thị chuỗi ra màn hình	Demo VGA

12	3	Đo đạc, so sánh với CPU và bản 0–9; viết báo cáo	Bảng so sánh độ chính xác, tài nguyên, độ trễ, công suất

11. Rủi ro và cách xử lý

Rủi ro	Cách xử lý
MLP không đạt độ chính xác (chữ giống nhau: O/0, I/1/l, S/5, Z/2)	Dùng EMNIST Balanced (đã gộp lớp dễ nhầm), QAT, tăng N_HID; nếu vẫn thiếu thì chuyển sang CNN nhỏ ở một giai đoạn riêng.
Dấu câu ít dữ liệu, dễ nhầm với ảnh trống	Thu nhiều mẫu, tăng cường dữ liệu (xoay, dịch, đổi độ dày nét); căn giữa và phóng to giống MNIST.
Hết BRAM trên Basys 3	Giảm N_HID xuống 96, hoặc dùng Nexys A7-100T (135 BRAM36).
Ảnh vẽ tay ngoài đời khác dữ liệu huấn luyện	Tiền xử lý trên PC giống hệt lúc huấn luyện (căn giữa, phóng to, độ dày nét).
Sai lệch Python ↔ RTL khi thêm lớp	Kiểm từng lớp riêng với `tb_fc_layer` trước khi ghép cả hệ thống.